Number System

09 June 2026 05:08

How are numeric data items actually stored in the computer memory?

How much space(memory locations) is allocated for each type of data?
- byte, short, int, long, float, double, char, boolean.

How are characters and strings are stored in memory?

Number System:: The basics

We are accustomed to using the so called **decimal number system.**
- Ten digits:: 0,1,2,3,4,5,6,7,8,9
- Every digit position has a weight which is a power of 10.
- Base or radix is 10

Example:

$234 = 2 * 10^2 + 3 * 10^1 + 4 * 10^0$

$250.67 = 2 * 10^2 + 5 * 10^1 + 0 * 10^0 + 6 * 10^{-1} + 7 * 10^{-2}$

Binary Number System

Two digits:
- **0 and 1**
- **Every digit position has a weight which is a power of 2.**
- **Base or radix is 2**

Example:

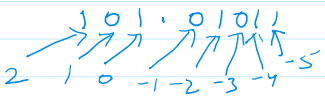
$110 = 1 * 2^2 + 1 * 2^1 + 0 * 2^0$

$110.01 = 1 * 2^2 + 1 * 2^1 + 0 * 2^0 + 0 * 2^{-1} + 1 * 2^{-2}$

Binary to decimal conversion

Each digit position of a binary number has a weight.
- **Some power of two**

every digit is b



A binary number:

$B = b_{n-1}b_{n-2}\ldots b_1 b_0 b_{-1}b_{-2}\ldots b_{-m}$

Corresponding value in decimal:

$$D = \sum_{i=-m}^{i=n-1} b_i \cdot 2^i$$

$$\sum_{i=-5}^{3-1=2} b_i \cdot 2^i$$

n = number of digits in integer part

Examples:

$10101011_2 = 1\times2^5 + 0\times2^4 + 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0$

$= 32 + 0 + 8 + 0 + 2 + 1$

$= 43_{10}$

$0.0101_2 = 0\times2^{-1} + 1\times2^{-2} + 0\times2^{-3} + 1\times2^{-4}$

$0.0101_2 = 0\times2^{-1} + 1\times2^{-2} + 0\times2^{-3} + 1\times2^{-4}$

$= 0.2500 + 0.0625$

$= 0.3125_{10}$

$101.11 = 5 + 1\times2^{-1} + 1\times2^{-2}$

$5 + 0.50 + 0.25 = 5.75_{10}$

Decimal-to-Binary Conversion

Consider the integer and fractional parts separately

For integer part,

- **Repeatedly divide the given number by two and go on accumulating the remainder until the number becomes zero.**
- **Arrange the reminders in reverse order.**

For the fractional part:

- **Repeatedly multiply the given fraction by two.**
  - **Accumulate the integer part (0, 1).**
  - **If the integer part is one, chop it off.**
- **Arrange the integer part in order they are obtained.**

Examples:

$(239.25)_{10} = (\ ?\ )_2$

2	239	
2	119	1
2	59	1
2	29	1
2	14	1
2	7	0
2	3	1
2	1	1
	0	1

$(239)_{10} = (11101111)_2$

2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
1024	512	256	128	64	32	16	8	4	2	1
			1	1	1	0	1	1	1	1

1023
0 to 511

365

1088

$111011010_1 = 949_{10}$

1 1 1 0 1 1 0 1 0 1

1 0 1 1 0 1 1 0 1

1 0 0 0 1 0 0 1 1 0 1

4 4 d

0.25×2

0 | 50×2

1 | 00

$239 + 0.25 = 11101111 + .01$

$(239.25)_{10} = (11101111.01)_2$

$1101110)_2 = 110$

$1101_{10} = (44d)_{12}$

Python code for conversion of numbers

```
bs = '1101'
print(int(bs,2)) ← binary to int
13
```

```
bs = '1101'
print(int(bs,2)) ← binary to int
13
print(oct(13)) ← int to octal
0o15
print(hex(13)) ← int to hex
0xd
print(bin(13)) ← int to binary
0b1101
```

Java

```
public class Coverersion
{
    public static void convertIntegerToBinary(int v){
        System.out.print("\nBinary equivalent of " + v +" is: " +
        Integer.toBinaryString(v));
        System.out.print("\n?: "+Integer.toUnsignedString(110, 2));

        System.out.print("\n ??: "+Integer.toString(1101, 16));
    }
}
```

int is converted to binary $(1101110)_2$

Methods available in Java for conversion:

```
String   toBinaryString(int)
String   toHexString(int)
String   toOctalString(int)
String   toString(int)
String   toString(int, int)
long     toUnsignedLong(int)
String   toUnsignedString(int)
String   toUnsignedString(int, int)
```

```
Integer.to
```

Hexadecimal Number System
A compact way of representing binary numbers
16 different symbols(radix = 16)

$16 = 2^4$ ← bits

0	0	0	0	0
0	0	0	1	1
0	0	1	0	2
0	0	1	1	3
0	1	0	0	4
0	1	0	1	5
0	1	1	0	6
0	1	1	1	7
1	0	0	0	8
1	0	0	1	9
1	0	1	0	A (10)
1	0	1	1	B (11)

1	0	1	0	A (10)
1	0	1	1	B (11)
1	1	0	0	C (12)
1	1	0	1	D (13)
1	1	1	0	E (14)
1	1	1	1	F (15)

Binary to Hexadecimal Conversion

For the integer part:
- **Scan the binary number from right to left.**
- **Translate each group of four bits into the corresponding hexadecimal digit.**
  - **Add leading zeros if necessary.**

For the fractional part:
- **Scan the binary number from left to right.**
- **Translate each group of four bits into the corresponding hexadecimal digit.**
  - **Add trailing zeros if necessary.**

For example:-

$(1011101101.01101110)_2 = (2ed.67)_{16}$

2 e d 6 7

$(0.10000100)_2 = (0.84)_{16}$

8	4	2	1	0 to 15
1	1	0	1	
1	1	1	0	
1	0	0	0	
0	1	0	1	
1	0	1	0	
		1	1	
1	1	0	1	
0	0	1	1	
0	0	1	0	

Hexadecimal to binary Conversion

Translate every hexadecimal digit into its 4 bit binary equivalent.

$(3A5)_{16} = (1110100101)_2$

$(12.3D)_{16} = (10010.00111101)_2$

Octal Number System
A compact way of representing binary numbers
8 different symbols(radix or base = 8)

0 0 0	→ 0
0 0 1	→ 1
0 1 0	→ 2
0 1 1	→ 3
1 0 0	→ 4
1 0 1	→ 5
1 1 0	→ 6
1 1 1	→ 7

Binary to Octal Conversion

For the integer part:

- **Scan the binary number from right to left**
- **Translate each group of three bits into a corresponding octal digit**
  - **Add leading zeros if necessary**

For the fractional part:

- **Scan the binary number from left to right**
- **Translate each group of three bits into the corresponding octal digit**
  - **Add trailing zeros if necessary**

For example:

$(101010101)_2 = (525)_8$

$(0.010101110)_2 = (0.256)_8$

$(00101.01101)_2 = (15.32)_8$

bit → switch → ON → 1, OFF → 0

Octal to binary Conversion

Translate every octal digit into its 3 bit binary equivalent.

3-bit switch

2^2	2^1	2^0
4	2	1
1	1	1
0	1	1
1	0	0
1	1	0
0	1	0

$(437)_8 = (100\ 011\ 111)_2$

$(53.26)_8 = (101011.010110)_2$

Decimal to octal

$(983)_{10} = (\ ?\ )_8$

8 | 983		
8 | 122	− 7	↑
8 | 15	− 2	
8 | 1	− 7	
0	− 1	

$(983)_{10} = (1727)_8$

$(8)_{10} = (\ ?\ )_8$

8 | 8	
8 | 1	− 0 ↑
0	− 1

$(8)_{10} = (10)_8$

1/2048 = 0.0005
1/64 = 0.0156

$(201.53)_{10} = (\ ?\ )_8$

For integer part-

8 | 201	
8 | 25	1 ↑

4	53 × 8
1	24 × 8
	92 × 8

$(31.4172)_8 = (\ ?\ )_{10}$

$= 3\times8^1 + 1\times8^0 + 4\times8^{-1} + 1\times8^{-2} + 7\times8^{-3} + 2\times8^{-4}$

$= 192 + 8 + 1 + \frac{4}{8} + \frac{1}{64} + \frac{7}{512} + \frac{2}{2048}$

8	201	
8	25	1
8	3	1
	0	3

4	24 X 8
1	92 X 8
7	36 X 8
2	88

$(201.53)_{10} \approx (3230.4172)_8$

1 2 4 8 16 32 64 512 1024 2048

201 + .5 + 0.0156 + 0.0137 + 0.0005

201.5298

0.0156 + 0.0137 + 0.0005 + 0.5 = 0.5298

octal to decimal.

Hexadecimal to decimal

$(ABC)_{16} = (?)_{10}$

1010 1011 1100

8	4	2	1
1	1	0	0
1	0	1	1
1	0	1	0

hex to bin → dec

4096	2048	1024	512	256	128	64	32	16	8	4	2	1
	1	0	1	0	1	0	1	1	1	0	0	

2048 + 512 + 128 + 32 + 16 + 8 + 4 = 2748

2048 + 512 + 128 + 32 + 16 + 8 + 4 = 2748

Unsigned binary Numbers

A n-bit binary number

$B = b_{n-1}\, b_{n-2}\, b_{n-3} \cdots b_2\, b_1\, b_0$

2^n distinct combinations are possible, 0 to $2^n - 1$

For example, for n=3, there are 8 distinct combination.

000, 001, 011, 100, 101, 110, 111

Range of numbers that can be represented.

n = 8	0 to $2^8 - 1$ (255)
n = 16	0 to $2^{16} - 1$ (65535)
n = 32	0 to $2^{32} - 1$ (4294967295)

n = 8

128	64	32	16	8	4	2	1	
0	0	0	0	0	0	0	0	→ 0
1	1	1	1	1	1	1	1	→ 255

unsigned integer = **0 to $2^n - 1$** ∵ n = number of bits

Signed Integer representation

Many of the numerical data items that are used in programs are signed (positive or negative).

Question:: How to represent sign?

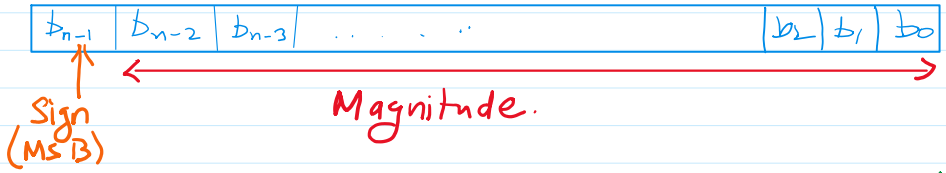
Three possible approaches:

i. Sign-magnitude representation
ii. One's complement representation
iii. Two's complement representation

i. Sign-magnitude representation

For an N - bit representation

- The most significant bit (MSB) indicates sign
  - 0 -> positive
  - 1 -> negative
- The remaining (N - 1) bits represents magnitude



$n = 8$

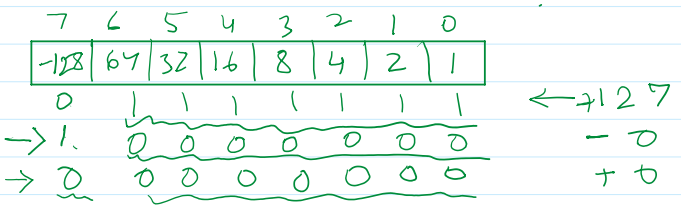
Range of the number that can be represented:

Maximum :: $+(2^{n-1}-1)$

Minimum :: $-(2^{n-1}-1)$

max $\Rightarrow 2^{8-1}-1 = 2^7-1 = 127$

min $\Rightarrow -1(2^{8-1}-1) = -1(2^7-1) = -127$



A problem:

Two different representation of zeros:

+0 -> 0 0000…0

-0 -> 1 0000…0